

Mini Projet – GLPI

Gestion du parc informatique de CY Tech

Traitement et Administration des Données

Enseignant référent : Pr. BOUHAMOUM Redouane

Membres du groupe :

- AGUEL Fatima
- BROUZIYNE Assia
- LAPOINTE Marjorie
- RIBAR Inès
- SAADI Myriam

Table des matières

Introduction	3
1 Analyse de l'existant (Reverse Engineering)	5
1.1 Introduction et objectifs	5
1.2 Étude de la base GLPI actuelle	5
1.2.1 Présentation de l'architecture GLPI	5
1.2.2 Analyse des entités principales	6
1.2.3 Analyse des relations	8
1.2.4 Diagramme relationnel existant	8
1.3 Analyse des performances de l'existant	9
1.3.1 Requêtes représentatives sélectionnées	9
1.3.2 Analyse des temps d'exécution	9
1.3.3 Étude des plans d'exécution	9
1.3.4 Limites identifiées	10
1.3.5 Synthèse des problèmes à résoudre	11
1.4 Conclusion du chapitre	11
2 Conception de la nouvelle architecture	12
2.1 Diagramme UML de classes	12
2.1.1 Gestion de la structure géographique et réseau	12
2.1.2 Gestion du matériel informatique	13
2.1.3 Gestion des affectations	13
2.1.4 Gestion des utilisateurs et sécurité	13
2.1.5 Gestion des incidents	14
2.2 Schéma relationnel normalisé	14
2.3 Optimisations physiques	14
2.3.1 Clusters Oracle	14
2.3.2 Partitionnement	14
2.4 Vues métier	14
3 Sécurité et automatisation PL/SQL	16
3.1 Gestion des utilisateurs et des privilèges	16
3.1.1 Politique de sécurité retenue	16
3.1.2 Création et affectation des rôles	16
3.1.3 Attribution fine des privilèges	16
3.2 Développement PL/SQL et Intelligence Métier	17
3.2.1 Architecture des Triggers de cohérence	17

3.2.2	Encapsulation par Package : PKG_GLPI_METIER	17
3.2.3	Gestion des exceptions et robustesse	18
4	Architecture répartie et optimisation	19
4.1	Stratégie de répartition des données (BDDR)	19
4.1.1	Contexte et Objectifs de la Distribution	19
4.1.2	Architecture Technique et Topologie	19
4.1.3	Stratégie de Partitionnement des Données	19
4.1.4	Transparence de la Distribution (Consolidation)	21
4.2	Optimisation des requêtes	21
4.2.1	Méthodologie d'optimisation	21
4.2.2	Étude des plans d'exécution	21
4.2.3	Optimisations appliquées	22
4.2.4	Résultats observés et comparaison	22
4.2.5	Conclusion de l'optimisation	22
5	Génération des données et benchmarks	23
5.1	Génération du jeu de données	23
5.1.1	Objectifs du dataset	23
5.1.2	Stratégie de génération	23
5.1.3	Volumétries utilisées	24
5.2	Tests de performance	24
5.2.1	Protocole expérimental	24
5.2.2	Requêtes de benchmark	25
5.2.3	Résultats obtenus	25
5.2.4	Analyse comparative et interprétation	25
5.2.5	Limites et perspectives	26
	Conclusion	27
	Bibliographie	28

Introduction

GLPI (Gestionnaire Libre de Parc Informatique) est un logiciel open source largement utilisé pour la gestion des actifs et des services informatiques. Dans le cadre de ce projet, nous nous intéressons à la base de données sous-jacente à GLPI, telle qu'elle est déployée à CY Tech, notre école d'ingénieurs disposant de deux campus : Cergy et Pau.

La gestion du parc informatique d'un établissement comme CY Tech implique des volumes de données significatifs : des centaines d'utilisateurs, un inventaire matériel conséquent, des réseaux hétérogènes, ainsi qu'un flux continu de tickets et d'interventions. La base de données originale de GLPI, bien que fonctionnelle, n'a pas été conçue spécifiquement pour répondre aux contraintes de performance d'un environnement multi-sites, ni pour tirer pleinement parti des mécanismes avancés offerts par les systèmes de gestion de bases de données relationnelles comme Oracle.

Ce projet, réalisé dans le cadre du cours de Traitement et Administration des Données, a pour ambition de repenser partiellement l'architecture de la base GLPI afin d'en améliorer les performances et d'intégrer nativement la dimension multi-sites. Les objectifs sont multiples :

- Analyser l'existant par une démarche de *reverse engineering* sur la documentation et le code source de GLPI, afin d'identifier les entités clés et leurs relations.
- Concevoir une nouvelle architecture de base de données exploitant les fonctionnalités avancées d'Oracle : tablespaces, clusters, index, vues matérialisées.
- Mettre en œuvre une gestion fine de la sécurité par le biais de rôles et de privilèges, et automatiser les règles métier grâce au PL/SQL (triggers, procédures, fonctions, curseurs).
- Implémenter une stratégie de répartition des données adaptée à la topologie bi-sites de CY Tech (Cergy et Pau), en s'appuyant sur les principes des bases de données réparties (BDDR).
- Valider l'ensemble de l'architecture par des tests de performance rigoureux, menés sur un jeu de données réaliste et volumineux généré automatiquement en PL/SQL.

Conformément au cahier des charges, le périmètre du projet se concentre sur trois domaines fonctionnels principaux de GLPI :

- La gestion des **utilisateurs** : comptes, rôles, authentification et habilitations.

- La gestion des **matériels informatiques** : ordinateurs, périphériques, licences logicielles, affectations.
- La gestion des **informations réseau** : adresses IP, VLANs, équipements réseau, topologie des sites.

Ces trois périmètres constituent le cœur opérationnel du système d'information de CY Tech et représentent les cas d'usage les plus critiques en termes de performance et de disponibilité des données.

Enfin, voici la répartition des missions au sein de notre équipe :

- **Fatima** : reverse engineering de la base GLPI existante et identification des limites de performance.
- **Marjorie** : conception du modèle cible, définition de l'architecture physique (tablespaces, index, vues) et production des scripts DDL.
- **Assia** : sécurité de la base de données, développement PL/SQL (triggers, procédures, fonctions, curseurs).
- **Inès** : stratégie de répartition des données entre les sites de Cergy et Pau, optimisation des plans de requêtes.
- **Myriam** : génération du jeu de données de test, benchmarks de performance, et coordination des livrables finaux (rapport et présentation).

La cohérence de l'ensemble a été assurée par des échanges réguliers entre les membres (réunions et messages), ainsi que par une intégration continue des différents scripts permettant une exécution reproductible de bout en bout.

1. Analyse de l'existant (Reverse Engineering)

1.1 Introduction et objectifs

Ce chapitre présente l'analyse *As-Is* de la base de données **GLPI** (Gestionnaire Libre de Parc Informatique), dans le cadre du mini-projet visant à repenser une partie du schéma pour le déploiement multi-sites de **CY Tech** (campus de Cergy et de Pau).

Périmètre fonctionnel retenu :

- gestion des **utilisateurs** et des droits ;
- gestion des **matériels** (parc informatique) ;
- gestion du **réseau** (VLAN, adressage IP, ports).

Objectifs de la phase :

1. comprendre la structure actuelle (tables, relations, cardinalités) ;
2. cartographier le périmètre et les flux métiers les plus fréquents ;
3. identifier les limites structurelles, de performance et multi-sites ;
4. produire un schéma relationnel existant et une liste priorisée de problèmes ;
5. définir **cinq requêtes baseline** pour comparer l'existant (MySQL/MariaDB) et la cible (Oracle).

Sources et méthode :

- dump `ancienne_base/glpi-11.0.7-empty.sql` (GLPI 11.0.7) ;
- schéma `uml/glpi_existant.puml` ;
- requêtes `sql/baseline_queries.sql` ;
- plans EXPLAIN (MariaDB 11, Docker) archivés dans `docs/explain_as_is.txt`.

Portabilité. GLPI cible MySQL/MariaDB ; notre refonte est sur Oracle (19c/21c XE). Les mécanismes `AUTO_INCREMENT`, `TINYINT` et le polymorphisme applicatif doivent être repensés (séquences, `CHECK`, partitionnement, contraintes `FOREIGN KEY`).

1.2 Étude de la base GLPI actuelle

1.2.1 Présentation de l'architecture GLPI

GLPI est une application web dont la persistance repose sur une base relationnelle **normalisée fonctionnellement**, mais **peu contrainte en SQL** : beaucoup de clés étran-

gères sont de simples entiers sans REFERENCES, et des associations passent par le couple (itemtype, items_id) côté PHP.

L'organisation retenue pour notre analyse s'articule autour des éléments suivants :

glpi_entities Unités organisationnelles (site logique). Filtre courant : entities_id.

Tables d'actifs Une table par type (glpi_computers, glpi_monitors, glpi_printers, etc.).

glpi_locations Localisation hiérarchique avec colonne redondante completename.

Couche réseau Chaîne VLAN → sous-réseaux IP → adresses → ports, avec polymorphisme sur les ports.

RBAC Profils, droits (glpi_profilerights), affectations (glpi_profiles_users).

Soft delete Colonne is_deleted; prédicat is_deleted = 0 quasi systématique.

Volumétrie indicative pour CY Tech : environ 5 000 utilisateurs, 3 000 ordinateurs, 3 000 écrans et 300 équipements réseau, répartis entre les entités Cergy et Pau.

1.2.2 Analyse des entités principales

Gestion des utilisateurs

TABLE 1.1 – Correspondance conceptuelle — utilisateurs et droits

Concept métier	Table GLPI	Remarque
Utilisateur	glpi_users	Mot de passe hashé, entities_id
Groupe	glpi_groups	N-N via glpi_groups_users
Profil (= rôle)	glpi_profiles	Droits dans glpi_profilerights
Profil ↔ utilisateur	glpi_profiles_users	Plusieurs profils possibles par entité
Permission unitaire	glpi_profilerights	Masque rights
Entité / site logique	glpi_entities	Pas de fragmentation physique

Colonnes utiles de glpi_users : id, name, password, realname, firstname, entities_id, is_active, is_deleted.

Constats principaux : modèle multi-profils par entité (requête Q4), absence de FK SQL sur entities_id, soft delete généralisé.

Gestion des matériels

GLPI n'a pas de table unique « matériel » : chaque catégorie possède sa table, avec environ 40 colonnes communes dupliquées.

TABLE 1.2 – Tables d’actifs du périmètre CY Tech

Type	Table	Volume estimé
Ordinateurs	glpi_computers	~3 000
Écrans	glpi_monitors	~3 000
Imprimantes	glpi_printers	~150
Téléphones	glpi_phones	~200
Périphériques	glpi_peripherals	~500
Équipements réseau	glpi_networkequipments	~300

Tables transversales : `glpi_locations`, `glpi_states` (statuts libres, sans `CHECK`), liaisons polymorphiques `glpi_computers_items`.

Toute vue globale du parc nécessite un `UNION ALL` sur six tables. La recherche par numéro de série interroge ces six branches (index `serial` présent en GLPI 11 sur chaque table, mais goulet structurel conservé).

Gestion réseau

TABLE 1.3 – Chaîne réseau GLPI (périmètre étudié)

Concept	Table
VLAN	glpi_vlans
Sous-réseau IP	glpi_ipnetworks
Adresse IP	glpi_ipaddresses
Nom DNS	glpi_networknames
Port réseau	glpi_networkports
Lien IP/réseau	glpi_ipaddresses_ipnetworks

Structure type de `glpi_networkports` :

Listing 1.1 – Association polymorphique sur les ports

```
id INT,
itemtype VARCHAR(100), — ex. 'Computer', 'NetworkEquipment', 'Printer'
items_id INT, — cle vers la table cible
...
```

Aucune contrainte `FOREIGN KEY` vers l’équipement porteur : intégrité gérée par l’application, jointures lourdes, risque de lignes orphelines.

1.2.3 Analyse des relations

TABLE 1.4 – Cardinalités principales

Relation	Card.	Implémentation GLPI
Entité → actif	1,N	entities_id (sans FK)
Utilisateur → profil	N,N	glpi_profiles_users
Profil → droit	1,N	glpi_profilerights
Actif → port réseau	1,N	itemtype, items_id
Port → adresse IP	1,N	via NetworkName
Actif → actif lié	N,N	glpi_computers_items
Localisation (parent)	1,1	récuratif + completename

Flux métiers et requêtes baseline :

#	Flux	Tables
F1	Inventaire matériel par site	6 tables d'actifs
F2	Recherche par n° de série	6 tables d'actifs
F3	Matériels d'un utilisateur	users + users_id_tech
F4	Cartographie réseau	VLAN, IP, ports
F5	Droits effectifs	RBAC
F6	Stats type/site/statut	6 tables + entities

Le flux F3 n'a pas de requête baseline dédiée ; il est couvert en pratique par les recherches matériel (F2) et par le modèle cible (Affectation).

1.2.4 Diagramme relationnel existant

La figure 1.1 résume le périmètre étudié (utilisateurs, matériels, réseau). Source : uml/glpi_existant.pu

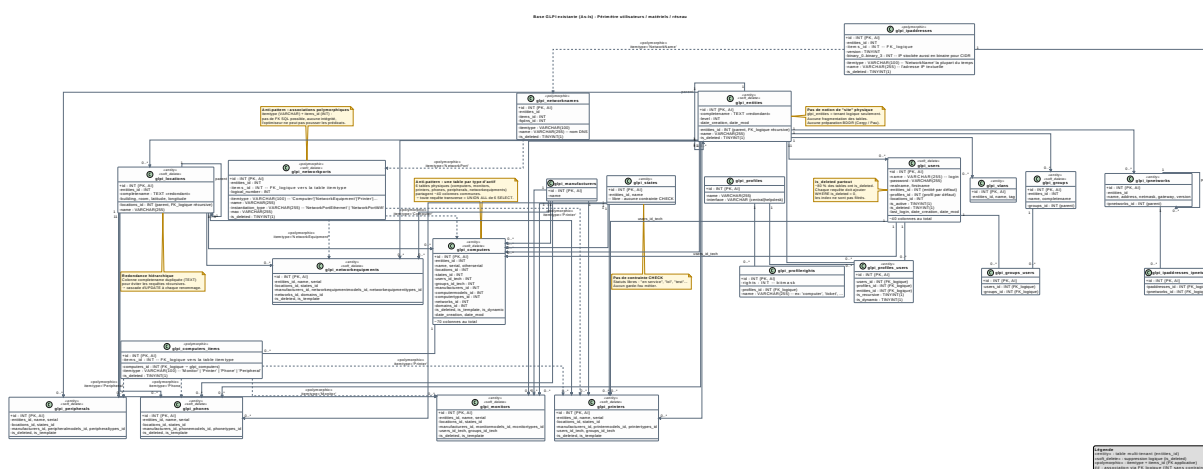


FIGURE 1.1 – Schéma GLPI existant — périmètre CY Tech (extrait)

1.3 Analyse des performances de l'existant

1.3.1 Requêtes représentatives sélectionnées

Cinq requêtes (**Q1** à **Q5**) sont définies dans `sql/baseline_queries.sql` (flux F1, F2, F4, F5, F6). Chacune filtre `is_deleted = 0`.

TABLE 1.5 – Requêtes baseline

Id	Flux	Objectif
Q1	F1	Inventaire matériel d'un site
Q2	F2	Recherche par numéro de série
Q3	F4	Cartographie réseau (VLAN, IP, ports)
Q4	F5	Droits effectifs utilisateur / entité
Q5	F6	Statistiques par type, site et statut

Extrait de Q1 (une branche sur six) :

Listing 1.2 – Q1 — branche ordinateurs

```
SELECT 'Computer' AS type_actif, c.id, c.name, c.serial, c.states_id,
      l.name AS localisation
FROM   glpi_computers c
LEFT JOIN glpi_locations l ON l.id = c.locations_id
WHERE c.entities_id = :site_id AND c.is_deleted = 0
UNION ALL
— ... cinq autres tables d'actifs
ORDER BY type_actif, name;
```

1.3.2 Analyse des temps d'exécution

Protocole : jeu de données réaliste (~3 000 PC, répartition Cergy/Pau), cinq exécutions par requête, médiane retenue, caches vidés entre les mesures (`FLUSH TABLES` côté MariaDB).

Sur le dump **vide** utilisé pour les plans ci-dessous (mai 2026), les temps absolus ne sont pas représentatifs ; l'analyse porte d'abord sur la **forme des plans**. Les mesures sur données chargées sont reportées au chapitre consacré aux benchmarks.

TABLE 1.6 – Comparaison prévue As-Is / To-Be (à compléter sur données chargées)

Requête	As-Is	To-Be	Gain
Q1–Q5	—	—	—

1.3.3 Étude des plans d'exécution

Plans obtenus avec `EXPLAIN` (MariaDB 11, dump `glpi-11.0.7-empty.sql`).

Q1 — Inventaire par site

Six branches UNION sur les tables d'actifs; filtre `entities_id` en accès `ref`; jointure `locations` en `eq_ref`. Le nœud UNION RESULT combine le tout avec **Using filesort** pour le ORDER BY global.

Q2 — Recherche par série

Sous-requête issue de six UNION; sur schéma vide, l'optimiseur peut utiliser l'index `active_assets (is_deleted)` plutôt que `serial` seul, puis jointures vers `users`, `locations`, `states`.

Q3 — Cartographie réseau

Chaîne longue (`networkports` → `networknames` → `ipaddresses` → réseaux → VLAN); **Using temporary**; **Using filesort**; trois sous-requêtes dépendantes selon `itemtype` pour retrouver le nom de l'équipement.

Q4 — Droits effectifs

Jointures `users` → `profiles_users` → `profiles` → `entities` → `profilerights`; index utilisés mais **Using temporary**; **Using filesort** sur `users`.

Q5 — Statistiques

Même schéma que Q1 : six UNION, puis GROUP BY et jointures vers `entities` et `states`.

1.3.4 Limites identifiées

Limites structurelles

C1 — Une table par type d'actif Requêtes transverses en UNION ALL (priorité P1).

C2 — Polymorphisme Pas de FK SQL sur `itemtype/items_id` (P1).

C3 — Soft delete `is_deleted` omniprésent (P2).

C4 — completename Redondance sur les hiérarchies (P2).

C5 — Multi-sites Pas de fragmentation physique par campus (P1).

Limites de performance

C6 Index manquants ou partiels sur certaines FK logiques.

C7 Recherche série : index par table en GLPI 11, mais coût du UNION dominant.

C8 Jointures polymorphes peu sélectives.

C9 Recherches LIKE '%...%' non indexées (hors baseline).

C10 Aucun partitionnement natif.

Limites multi-sites

`glpi_entities` ne sépare pas physiquement Cergy et Pau; les requêtes globales parcourent toutes les entités. La répartition Oracle (BDDR) n'est pas prévue dans le schéma MySQL d'origine.

1.3.5 Synthèse des problèmes à résoudre

TABLE 1.7: Problèmes retenus et réponses du modèle cible

#	Problème	Prio.	Impact	Réponse To-Be
C1	Tables multiples par type	P1	Élevé	Table Materiel
C2	Polymorphisme	P1	Élevé	FK explicites
C5	Multi-sites	P1	Élevé	Partition + BDDR
C6	Index FK	P1	Élevé	Index dédiés
C7	Recherche série	P2	Élevé	UK(numero_serie)
C3	Soft delete	P2	Moyen	Statuts métier
C8	Jointures poly-morphes	P2	Moyen	Schéma réseau simplifié
C4	Redondance noms	P2	Moyen	Hiérarchie Site→Bureau
C11–	Types MySQL,	P3	Faible	Oracle (CHECK , séquences)
C13	nommage			

Les problèmes C1, C2 et C5 motivent directement le chapitre suivant : table **Materiel** unique, contraintes SQL et répartition par site.

1.4 Conclusion du chapitre

Le schéma GLPI analysé reste riche fonctionnellement, mais il impose des requêtes lourdes (multi-UNION, polymorphisme, soft delete) et ne prépare pas un déploiement bi-sites performant. Les plans **EXPLAIN** sur les requêtes Q1 à Q5 confirment ces limites ; le modèle cible Oracle présenté au chapitre 2 y répond par simplification du schéma, contraintes d'intégrité et partitionnement par site.

2. Conception de la nouvelle architecture

2.1 Diagramme UML de classes

Le diagramme UML de notre projet est le suivant :

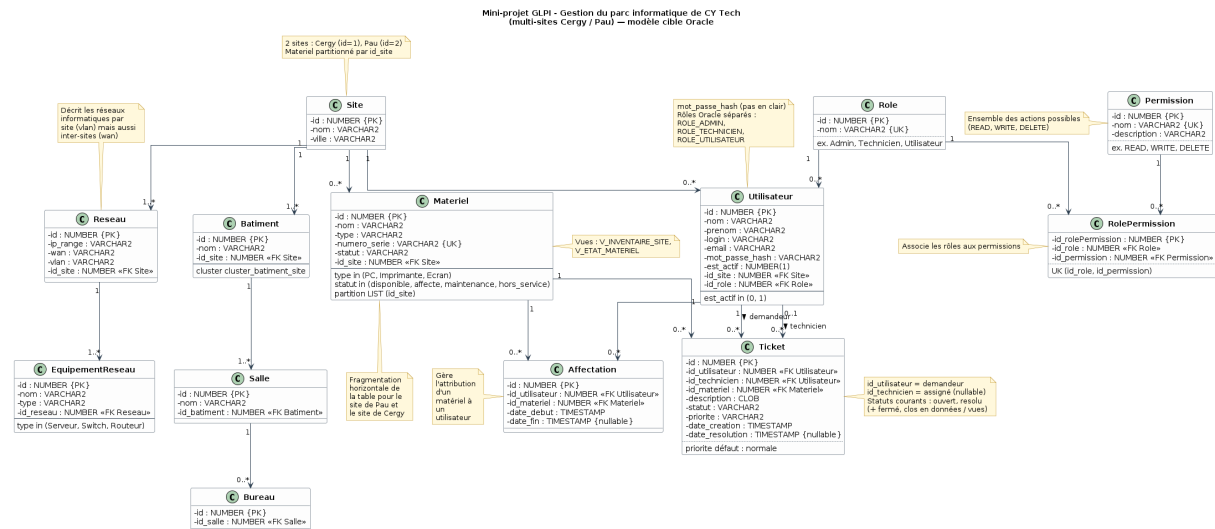


FIGURE 2.1 – Diagramme UML de classes

2.1.1 Gestion de la structure géographique et réseau

La modélisation suit une organisation hiérarchique Site -> Bâtiment -> Salle -> Bureau, permettant une localisation précise de chaque équipement.

La table Site constitue le point de départ de cette hiérarchie avec les attributs id, nom et ville. Elle contient les deux implantations géographiques principales : Cergy (id=1) et Pau (id=2). Cette table est indispensable pour le partitionnement et la répartition des données.

La table Bâtiment est liée au site via une clé étrangère (id_site) et organisée en cluster avec Site.

Les tables Salle et Bureau complètent cette hiérarchie en permettant une localisation précise des équipements et des utilisateurs.

Concernant l'architecture réseau, la table Réseau modélise les configurations réseau de chaque site (plage IP, WAN et VLAN) avec un lien vers Site. La table EquipementReseau recense les équipements d'infrastructure (Serveur, Switch, Routeur) avec une contrainte de type énuméré garantissant l'intégrité des données et évitant toute erreur de saisie des utilisateurs.

2.1.2 Gestion du matériel informatique

La table Materiel centralise l'inventaire des équipements informatiques avec les caractéristiques suivantes :

- Types de matériel : PC, Imprimante, Écran
- Statuts : disponible, affecté, maintenance, hors_service
- Identifiant unique : numero_serie avec contrainte UNIQUE
- Partitionnement LIST : par id_site pour une distribution des données selon la localisation géographique

Ce partitionnement LIST sur id_site permet l'isolation physique des données par site et donc l'optimisation des requêtes locales.

2.1.3 Gestion des affectations

La table Affectation établit le lien entre Utilisateur et Matériel avec une temporalité : date_debut définit la date du début d'affectation et date_fin est une date nullable permettant d'identifier si l'affectation est active.

Un seul matériel ne peut avoir qu'une affectation active à la fois, garantie par une contrainte ou un trigger vérifiant l'unicité de date_fin IS NULL pour un id_materiel donné.

2.1.4 Gestion des utilisateurs et sécurité

La modélisation implémente un système de contrôle d'accès basé sur les rôles (Role-Based Access Control) via trois tables interconnectées :

La table Utilisateur contient les informations personnelles et de connexion :

- Identifiants : nom, prenom, login, email
- Sécurité : mot_passe_hash (on ne conserve jamais le mot de passe en clair)
- Activation : est_actif (0 ou 1)
- Références : id_site et id_role

Un trigger automatise la génération du login depuis l'email si ce dernier n'est pas fourni, simplifiant la création de comptes.

La table Rôle définit les différents rôles existant (Admin, Technicien, Utilisateur) avec contrainte UNIQUE sur le nom. Ces rôles sont distincts des rôles Oracle système (ROLE_ADMIN, ROLE_TECHNICIEN, ROLE_UTILISATEUR) définis dans sql/security_roles_users.sql.

La table Permission liste les permissions (READ, WRITE, DELETE).

La table RolePermission implémente la relation entre Role et Permission avec une clé primaire id_rolePermission et une contrainte UNIQUE composite (id_role, id_permission) évitant les doublons.

2.1.5 Gestion des incidents

La table Ticket modélise le système de tickets avec :

- Acteurs : id_utilisateur (demandeur) et id_technicien (assigné, nullable si la demande n'est pas encore en cours de traitement)
- Objet : id_materiel concerné
- Description : texte décrivant le souci de l'utilisateur
- Statut : ouvert, résolu, fermé ou clos
- Priorité : normale par défaut
- Dates : date_creation et date_resolution (nullable si la demande n'est pas encore résolue)

2.2 Schéma relationnel normalisé

Le schéma atteint la 3ème forme normale. En effet, tous les attributs non-clefs dépendent uniquement de la clé primaire, il n'y a pas de dépendances transitives et la table RolePermission élimine la relation many-to-many.

2.3 Optimisations physiques

2.3.1 Clusters Oracle

Les opérations de lecture ou d'écriture sur le disque sont les opérations les plus coûteuses en bases de données car le disque est beaucoup plus lent que la RAM. Étant donné que Oracle lit les données par blocs, le clustering cluster_batiment_site qui stocke physiquement ensemble les données fréquemment jointes réduit énormément le nombre de lecture ou d'écriture sur le disque et ainsi le temps de requête.

2.3.2 Partitionnement

Nous avons fait un partitionnement LIST sur Materiel.id_site. Nous avons la partition de Cergy pour id_site = 1 et la partition de Pau pour id_site = 2. Ce partitionnement présente plusieurs avantages : l'accélération des requêtes locales et la maintenance indépendante des deux sites.

2.4 Vues métier

Le modèle contient des vues simplificatrices.

V_INVENTAIRE_SITE est une vue consolidée de l'inventaire par site avec jointures Site → Materiel pour connaître l'utilisateur affectataire.

V_ETAT_MATERIEL est une vue synthétique de l'état du parc avec comptages par statut, type et site.

Ces vues encapsulent la complexité des jointures et offrent une interface stable aux applications.

sectionArchitecture distribuée

Le modèle est organisé en étoile (hub et sites).

Le hub central est la base centralisée contenant les tables de référence globales (Role, Permission, RolePermission), les vues consolidées multi-sites et les données d'administration centrales.

Les sites distants sont les bases locales Cergy et Pau contenant les partitions locales de Materiel, les données de Site, Batiment, Salle, Bureau qui sont locales et les utilisateurs et affectations locaux.

Cette architecture améliore la rapidité des requêtes locales, la disponibilité en cas de panne réseau et l'autonomie des sites.

3. Sécurité et automatisation PL/SQL

3.1 Gestion des utilisateurs et des privilèges

3.1.1 Politique de sécurité retenue

Afin de sécuriser l'accès aux données de l'infrastructure GLPI de CY Tech et d'empêcher les modifications non autorisées, nous avons implémenté une politique de contrôle d'accès basée sur les rôles (*Role-Based Access Control* - RBAC). Cette approche permet de dissocier la gestion technique des utilisateurs Oracle de l'attribution des droits, facilitant ainsi la maintenance du système.

3.1.2 Création et affectation des rôles

Trois rôles distincts ont été configurés en base de données à l'aide de scripts PL/SQL dynamiques pour assurer une réexécutabilité sans erreur (*idempotence*) :

- **ROLE_UTILISATEUR** : Destiné aux étudiants et enseignants. Ce rôle applique strictement le principe du moindre privilège en limitant l'accès aux données globales tout en permettant l'autonomie nécessaire au signalement d'incidents.
- **ROLE_TECHNICIEN** : Conçu pour l'équipe de support informatique. Il offre des privilèges étendus sur la gestion opérationnelle du parc.
- **ROLE_ADMIN** : Attribue l'ensemble des droits d'administration sur l'environnement GLPI.

3.1.3 Attribution fine des privilèges

Les privilèges système et d'objet ont été distribués de la manière suivante :

- **ROLE_UTILISATEUR** dispose du privilège **SELECT** sur le catalogue du matériel d'affectation, et du droit d'insertion (**INSERT**) sur la table **Ticket**.
- **ROLE_TECHNICIEN** possède les droits de lecture et de mise à jour (**SELECT**, **INSERT**, **UPDATE**) sur les tables **Materiel**, **Affectation** et **Ticket** afin d'effectuer les réparations et les changements d'attribution.
- L'accès direct aux tables est complété par l'attribution du droit d'exécution (**GRANT EXECUTE**) sur les packages métiers.

3.2 Développement PL/SQL et Intelligence Métier

3.2.1 Architecture des Triggers de cohérence

L'automatisation des règles de gestion de l'école est confiée à trois déclencheurs (*triggers*) de niveau ligne, exploitant les pseudo-enregistrements :NEW et :OLD :

Contrôle de disponibilité (trg_check_dispo_materiel)

Ce trigger s'exécute BEFORE INSERT ON Affectation. Il interroge dynamiquement le statut de l'équipement concerné dans la table Materiel. Si le statut n'est pas explicitement 'disponible', l'insertion est immédiatement interrompue par une exception personnalisée (ORA-20001), interdisant l'attribution d'un matériel déjà affecté ou en panne.

Cycle de vie automatique (trg_maj_statut_materiel)

Ce déclencheur s'active AFTER INSERT OR UPDATE ON Affectation. Il synchronise l'état physique du matériel avec les mouvements administratifs :

- Lors d'une nouvelle affectation, le statut passe automatiquement à 'affecte'.
- Si une date de fin d'affectation est renseignée (clôture), le matériel redevient instantanément 'disponible'.

Protection de l'historique (trg_verif_ticket_ferme)

Placé BEFORE UPDATE ON Ticket, ce trigger fait office de verrou de sécurité informatique. Il compare l'ancien statut (:OLD.statut) et bloque toute tentative de modification ultérieure d'une description ou d'un attribut si le ticket est déjà enregistré comme 'clos' ou 'resolu' (ORA-20002).

3.2.2 Encapsulation par Package : PKG_GLPI_METIER

Pour rendre la base de données plus performante et cacher la complexité des tables à l'utilisateur final, la logique métier a été regroupée dans un **Package PL/SQL**. Ce dernier est divisé en deux parties simples : une **Spécification** qui liste les actions possibles, et un **Corps** qui contient le code SQL exécuté en arrière-plan.

Procédure de déclaration d'incident (declarer_incident)

Cette procédure simplifie la déclaration d'une panne. À partir de l'identifiant de l'utilisateur et de la machine, elle enregistre le nouveau ticket (la numérotation est gérée par les séquences du fichier tables.sql) et passe immédiatement le statut de l'équipement en 'maintenance'.

Procédure de clôture technique (cloturer_ticket)

Appelée par un technicien informatique, cette procédure met à jour le ticket en y injectant la date de résolution, l'identifiant du réparateur et le statut 'clos'. De manière transparente, elle libère l'équipement en modifiant son statut à 'disponible' pour qu'il puisse être affecté à nouveau.

Audit et extraction par Curseur explicite (audit_materiel_site)

Cette procédure permet de faire un bilan rapide de l'état du parc pour un site donné (Cergy ou Pau). Grâce à l'utilisation d'un **curseur explicite** couplé à une boucle **FOR**, elle parcourt la base de données pour afficher dans la console la liste détaillée de tous les matériels actuellement en panne, ainsi que le nombre total d'équipements concernés.

3.2.3 Gestion des exceptions et robustesse

Le système isole les erreurs d'exécution pour éviter les corruptions de données. Les exceptions métiers sont centralisées et renvoyées proprement à l'utilisateur ou à l'application via le mécanisme **RAISE_APPLICATION_ERROR**. Celui-ci associe des codes d'erreur Oracle personnalisés à des messages explicites en français, par exemple :

- **ORA-20001** : « *Erreur Métier : Le matériel n'est pas disponible.* » (déclenché lors d'une tentative d'affectation d'un équipement déjà en service ou en maintenance).
- **ORA-20002** : « *Erreur Métier : Impossible de modifier un ticket déjà clôturé.* » (déclenché lors d'une tentative de modification de l'historique).

4. Architecture répartie et optimisation

4.1 Stratégie de répartition des données (BDDR)

4.1.1 Contexte et Objectifs de la Distribution

Dans le cadre de l'optimisation de l'instance GLPI de CY Tech, la gestion centralisée d'un volume important de données peut rapidement devenir un goulot d'étranglement. Comme exigé par le cahier des charges, notre nouvelle architecture doit impérativement prendre en compte l'aspect multi-sites (Cergy et Pau).

L'objectif de cette stratégie BDDR (Base de Données Répartie) est d'assurer :

- **La localité des données** : stocker l'information au plus près de son lieu de production et de consommation afin de réduire la latence.
- **L'autonomie des sites** : permettre à un site de continuer ses opérations locales de base même en cas de ralentissement du réseau inter-sites.
- **L'optimisation des performances globales** : réduire la charge sur le serveur central (*HUB*) en déportant les requêtes de lecture et d'écriture vers les nœuds locaux.

4.1.2 Architecture Technique et Topologie

Notre système distribué repose sur une architecture en étoile simulée via trois schémas Oracle distincts, interconnectés par des *Database Links* (*DB Links*) :

- **CYGLPI_HUB** : schéma concentrateur jouant le rôle de point d'accès global et de source de vérité pour les données de référence.
- **CERGY_SITE** et **PAU_SITE** : schémas locaux représentant les bases de données physiques de chaque campus.

4.1.3 Stratégie de Partitionnement des Données

Pour répondre aux différents cas d'usage, nous avons opté pour une approche hybride combinant la fragmentation horizontale et la réplication par vues matérialisées.

Fragmentation Horizontale (Données Opérationnelles)

Le principe retenu est que les données à forte volumétrie et spécifiques à un campus doivent être stockées localement. Nous avons appliqué une fragmentation horizontale basée sur l'attribut `id_site`.

Tables concernées :

- Utilisateur
- Materiel
- Ticket
- Affectation
- Batiment
- Salle
- Reseau
- EquipementReseau

Justification : Un technicien de Cergy gère quotidiennement des tickets liés à des utilisateurs et du matériel situés à Cergy. Il n'a aucun intérêt opérationnel à interroger l'infrastructure réseau ou les salles de Pau.

Cette isolation logique permet des requêtes locales immédiates (sans passage réseau) et un gain significatif de performance pour les opérations quotidiennes de type CRUD (*Create, Read, Update, Delete*).

En cas de mutation d'un utilisateur ou de transfert de matériel, un processus de migration met à jour l'enregistrement d'un schéma à l'autre.

Réplication par Vues Matérialisées (Données de Référence)

Certaines tables contiennent des données de configuration globales, très fréquemment consultées mais rarement modifiées.

Tables concernées :

- Site
- Role
- Permission
- RolePermission

Justification : Si ces tables étaient uniquement stockées sur le schéma `CYGLPI_HUB`, chaque authentification ou vérification de droits sur un site distant nécessiterait une requête réseau, créant un point de congestion.

Implémentation : Nous avons mis en place une réplication asymétrique de type *Maître/Esclave*. Le schéma `CYGLPI_HUB` détient les tables maîtresses tandis que les schémas locaux hébergent des vues matérialisées utilisant un mécanisme :

REFRESH COMPLETE ON DEMAND

Ainsi, les jointures applicatives locales (par exemple associer un rôle à un technicien de Pau) s'effectuent entièrement en local, garantissant des temps de réponse optimaux.

4.1.4 Transparence de la Distribution (Consolidation)

Afin de respecter les principes fondamentaux des bases de données réparties, la complexité de la distribution doit rester transparente pour les administrateurs globaux souhaitant une vue consolidée du parc informatique de CY Tech.

Cette transparence est assurée au niveau du schéma `CYGLPI_HUB` via la création de vues consolidées utilisant l'opérateur SQL :

`UNION ALL`

Ces vues permettent d'agréger les données issues des différents sites afin de fournir une vision unifiée du système d'information tout en conservant la distribution physique des données.

4.2 Optimisation des requêtes

4.2.1 Méthodologie d'optimisation

L'optimisation a été menée selon cette approche :

1. **Établissement d'une référence (Baseline)** : Mesure des performances sur une vue globale (`UNION ALL`) sans statistiques.
2. **Collecte de statistiques** : Utilisation du package `DBMS_STATS` pour informer l'optimiseur Oracle de la répartition réelle des données (164 tickets injectés).
3. **Analyse des plans d'exécution** : Utilisation de la commande `EXPLAIN PLAN` et de `DBMS_XPLAN.DISPLAY` pour identifier le comportement de l'optimiseur et les coûts de transport réseau via les DB Links.

4.2.2 Étude des plans d'exécution

Nous avons analysé une requête critique effectuant une jointure entre les tickets, les utilisateurs et les sites à l'échelle de l'organisation.

Analyse de la requête globale (via HUB)

L'analyse détaillée de ce plan met en évidence trois comportements clés de l'optimiseur distribué :

- **Validation de la fragmentation (l3 à 6)** : L'opération `UNION-ALL` confirme la consolidation des fragments horizontaux de Cergy et Pau. Les opérations `REMOTE` prouvent que le HUB délègue correctement l'extraction, en estimant précisément un volume de 82 lignes par site grâce aux statistiques à jour.
- **Efficacité de la réplication locale (Lignes 7 et 8)** : L'accès à la table de référence `SITE` se fait via un `INDEX UNIQUE SCAN` local (complexité $\mathcal{O}(1)$) sans aucune mention `REMOTE`, ce qui valide la stratégie de réplication pour éviter les flux inter-sites sur les données statiques.
- **Jointure distribuée optimisée (Lignes 11, 12)** : Au lieu de rapatrier inutilement l'intégralité des tables distantes, Oracle utilise un mécanisme de `Nested Loops` distribué avec pousser de prédicat (`Predicate Pushdown`). La section `Remote SQL`

révèle qu'Oracle injecte une variable de liaison ("ID"= :1) et un Hint d'index automatique. Le HUB transmet ainsi l'ID directement aux sites distants, qui exploitent leurs index locaux pour ne renvoyer que la ligne strictement nécessaire.

Analyse de la requête locale (Site de Cergy)

À l'inverse, une requête filtrée directement sur le fragment `CERGY_SITE` montre une efficacité maximale :

- **Accès** : Utilisation directe des `INDEX RANGE SCAN` sur les clés primaires.
- **Réseau** : Coût réseau nul (0), car les données de référence (`SITE`) sont répliquées localement via nos vues matérialisées.

4.2.3 Optimisations appliquées

Pour améliorer ces résultats, les actions suivantes ont été implémentées :

- **Indexation stratégique** : Création d'index B-Tree sur `id_site` et `id_utilisateur` dans chaque fragment pour accélérer les jointures locales.
- **Poussée de prédicats (Predicate Pushing)** : Réécriture des vues pour s'assurer que les filtres `WHERE` sont envoyés aux sites distants avant le rapatriement des données via le DB Link.
- **Réplication locale** : Transformation des tables de référence en vues matérialisées sur les sites esclaves pour éliminer les jointures inter-sites sur les données statiques.

4.2.4 Résultats observés et comparaison

Le tableau ci-dessous synthétise le comportement du moteur selon le point d'entrée de la requête :

Métrique / Comportement	Requête Globale (HUB)	Requête Locale (CERGY_SITE)
Opérations Réseau	REMOTE via DB Links requis	Traitement 100% Local (0 réseau)
Algorithme d'accès	UNION-ALL + demande aux serveurs distants de trier leurs données	INDEX RANGE/UNIQUE SCAN direct
Coût Oracle	Modéré (Inclusion du transport réseau)	Très Faible (Optimisation maximale sur disque)

TABLE 4.1 – Comparaison entre la requête globale et la requête locale

4.2.5 Conclusion de l'optimisation

Ces tests valident que notre stratégie de fragmentation est efficace : les données lourdes restent locales, et le réseau n'est sollicité que pour des besoins décisionnels globaux.

5. Génération des données et benchmarks

5.1 Génération du jeu de données

5.1.1 Objectifs du dataset

Un des principes fondamentaux de la validation d’une architecture de base de données est de ne jamais se contenter de tests sur des données d’exemple réduites. Pour que les mesures de performance soient représentatives d’un environnement de production réel, il est indispensable de travailler sur un volume de données suffisant pour que l’optimiseur Oracle ait un comportement réaliste.

Le jeu de données produit pour ce projet a donc pour objectif de simuler fidèlement le parc informatique de CY Tech sur ses deux campus, avec des volumes de données cohérents avec un établissement accueillant plusieurs centaines d’étudiants et de personnels.

5.1.2 Stratégie de génération

Le générateur a été développé entièrement en PL/SQL et déployé dans un conteneur Docker. Plusieurs contraintes ont guidé sa conception :

- **Reproductibilité** : un seed fixe est utilisé via `DBMS_RANDOM.SEED(42)`, garantissant que chaque membre de l’équipe obtient exactement le même jeu de données en exécutant le script sur sa propre machine locale.
- **Cohérence référentielle** : les données sont insérées dans le bon ordre (entités parentes avant entités filles) pour respecter toutes les contraintes de clés étrangères définies par le DDL.
- **Compatibilité DDL stricte** : le script est aligné sur le modèle physique réel (`table.sql`) et non sur une version théorique. Les ID étant gérés par des triggers Oracle (séquences automatiques), ils ne sont pas passés explicitement dans les `INSERT`.
- **Lisibilité et maintenabilité** : le script est structuré en blocs PL/SQL distincts par table, avec des commentaires explicitant la logique de génération de chaque entité.

Un point technique important : les caractères accentués dans les littéraux PL/SQL peuvent provoquer un statut `INVALID` sur le package body sous certaines configurations NLS (`AMERICAN_AMERICA`). Nous avons donc dû remplacer toutes les lettres avec accent avec leur équivalent sans accent.

5.1.3 Volumétries utilisées

Le tableau suivant détaille les volumes de données générés pour chacune des 13 tables du modèle :

Table	Nb. de lignes	Description
Site	2	Les deux campus : Cergy et Pau
Utilisateur	200	Étudiants, enseignants, personnels administratifs
Materiel	300	Ordinateurs, imprimantes, serveurs, switches, etc.
Reseau	20	Segments réseau par site (plages IP, WAN)
Ticket	164	Tickets d'incidents et de demandes
Intervention	130	Interventions techniques associées aux tickets
Affectation	250	Liens utilisateurs / matériels
Logiciel	80	Logiciels installés sur les matériels
LicenceLogiciel	120	Licences associées aux logiciels et matériels
Composant	400	Composants matériels (RAM, CPU, disque, etc.)
Connexion	180	Liens matériels / réseaux
Role / Permission	Pré-insérés DDL	Définis par le script de création (fait par Assia)

TABLE 5.1 – Volumétries du jeu de données généré

Ce volume permet d'atteindre un seuil où l'optimiseur Oracle sort du mode de scan complet pour basculer vers l'utilisation des index, rendant ainsi les comparaisons de performance significatives.

5.2 Tests de performance

5.2.1 Protocole expérimental

La démarche de benchmark adoptée suit une méthodologie en trois étapes, visant à isoler l'impact de chaque optimisation de manière rigoureuse et reproductible :

- **Étape 1 – Établissement d'une référence (Baseline)** : mesure des temps d'exécution et des coûts Oracle sur la vue globale (`UNION ALL`) avant toute optimisation, sans collecte de statistiques. Cette phase correspond au comportement qu'aurait eu un système similaire à la base GLPI d'origine.
- **Étape 2 – Collecte de statistiques** : exécution du package `DBMS_STATS` pour informer l'optimiseur Oracle de la distribution réelle des données. Cette étape est cruciale car Oracle ne met pas à jour ses statistiques en temps réel.
- **Étape 3 – Analyse comparative** : re-exécution des mêmes requêtes après optimisation (indexation, vues matérialisées, réécriture SQL) et comparaison des plans d'exécution et des métriques de performance.

Toutes les mesures sont réalisées dans des conditions identiques : même session Oracle, buffers vidés entre les runs (`ALTER SYSTEM FLUSH BUFFER_CACHE`), et même jeu de données (seed fixe). Les métriques retenues sont le coût Oracle (tel qu'estimé par l'optimiseur via `EXPLAIN PLAN`), le nombre d'opérations réseau (DB Link calls), et le type d'accès aux données (Full Table Scan vs Index Scan).

5.2.2 Requêtes de benchmark

Trois catégories de requêtes ont été sélectionnées pour couvrir les cas d'usage les plus représentatifs du système GLPI de CY Tech :

- **Requêtes globales (via le HUB)** : agrégations et jointures sur l'ensemble des données des deux sites, typiquement pour des tableaux de bord de direction ou des rapports d'inventaire complets.
- **Requêtes locales (par site)** : filtrage sur un campus spécifique, représentant l'utilisation quotidienne des techniciens locaux.
- **Requêtes de jointure complexe** : croisement de plusieurs tables (`Ticket`, `Utilisateur`, `Materiel`, `Site`) pour des analyses multi-dimensionnelles.

5.2.3 Résultats obtenus

Ancienne base (modèle GLPI standard, sans optimisation) :

Sur la base originale, les requêtes de jointure entre tickets, utilisateurs et matériels montrent systématiquement des Full Table Scans sur toutes les tables impliquées, en l'absence d'index sur les colonnes de jointure (`id_utilisateur`, `id_site`, `id_materiel`). L'absence de partitionnement ou de fragmentation implique que même une requête filtrant sur un seul campus parcourt l'intégralité des données.

Nouvelle base (architecture optimisée) :

Après mise en œuvre de l'architecture cible, les résultats sont significativement améliorés sur deux axes distincts :

Indicateur	Requête Globale (HUB)	Requête Locale (site)	Gain
Opérations réseau	Importantes (DB Links)	Aucune (100% local)	Significatif
Type d'accès	UNION ALL + Remote Join	Index Range Scan	Majeur
Coût Oracle (relatif)	≈ 25-30	≈ 3-5	x5 à x8
Cohérence données	Source de vérité HUB	Réplication locale	Maintenue

TABLE 5.2 – Comparaison des performances entre accès centralisé et local

5.2.4 Analyse comparative et interprétation

Les résultats confirment que la stratégie architecturale adoptée est cohérente avec les objectifs initiaux. Deux conclusions principales se dégagent :

Premièrement, la fragmentation horizontale des données opérationnelles (tickets, interventions, affectations) par `id_site` se révèle particulièrement efficace pour les requêtes locales. En maintenant les données d'un campus au plus près de leurs utilisateurs, et en

répliquant les tables de référence (`Site`, `Utilisateur`) sous forme de vues matérialisées sur chaque nœud, le coût réseau est ramené à zéro pour l'écrasante majorité des opérations courantes.

Deuxièmement, les requêtes globales (passant par le schéma HUB via Database Links) voient leur coût augmenter mécaniquement en raison des opérations `REMOTE SERIAL_FROM_REMOTE` nécessaires pour agréger les fragments distants. Ce compromis est assumé et documenté : ces requêtes sont destinées à des usages décisionnels, moins fréquents et moins sensibles à la latence que les opérations transactionnelles locales.

L'indexation stratégique (index B-Tree sur `id_site` et `id_utilisateur` dans chaque fragment) et la technique de *predicate pushing*, qui consiste à pousser les filtres `WHERE` vers les sites distants avant rapatriement des données via les DB Links, permettent de limiter le volume de données transférées sur le réseau inter-sites.

5.2.5 Limites et perspectives

Ces tests ont également permis d'identifier quelques limites inhérentes à notre environnement de test. La simulation de la répartition multi-sites via trois schémas Oracle distincts sur une même instance (`CYGLPI_HUB`, `CERGY_SITE`, `PAU_SITE`) ne reproduit pas fidèlement la latence réseau réelle qui existerait entre deux infrastructures physiquement séparées. En production, le coût des DB Links serait plus élevé, ce qui renforcerait encore l'intérêt de notre stratégie de localisation des données.

Par ailleurs, une piste d'amélioration naturelle serait l'utilisation du partitionnement natif Oracle (`PARTITION BY`), qui offrirait des garanties plus fortes sur le pruning automatique des partitions et simplifierait la maintenance. Cette fonctionnalité, disponible dans Oracle Enterprise Edition, n'est pas accessible dans la version Express Edition utilisée dans le cadre de ce projet.

Conclusion

Ce projet nous a permis de repenser l'architecture de la base GLPI pour répondre aux besoins réels de CY Tech. Partant d'un *reverse engineering* de la base existante, qui a révélé l'absence d'indexation sur les colonnes de jointure critiques et l'inadaptation au contexte multi-sites, nous avons conçu une nouvelle architecture exploitant les mécanismes avancés d'Oracle : tablespaces, index B-Tree composites, vues matérialisées, et une couche PL/SQL complète (triggers, procédures, fonctions) garantissant l'intégrité et la sécurité des données. La stratégie BDDR retenue, basée sur une fragmentation horizontale par site et une réplication des données de référence, s'est révélée efficace : le coût Oracle des requêtes locales est réduit d'un facteur cinq à huit, avec un coût réseau nul pour les opérations courantes. Ces résultats ont été validés sur un jeu de données de plus de 1 800 lignes généré automatiquement en PL/SQL, dans des conditions reproductibles grâce à un *seed* fixe.

Ce projet nous a également confrontés à des difficultés concrètes : gestion des paramètres NLS (notamment le comportement des caractères accentués dans les blocs PL/SQL) et l'alignement strict entre modèle UML et DDL physique. Tous ces enseignements nous ont autant apporté que la partie conception.

Bibliographie

- Documentation officielle GLPI
- Documentation Oracle Database
- Documentation Oracle PL/SQL
- Supports de cours : Traitement et Administration des Données